Exercice : programmer dans un langage ésotérique

Objectif

Comprendre ce que signifie programmer dans un langage très éloigné du problème à résoudre, et ce qui rend cette tâche difficile, pour un humain comme pour un LLM.

Consigne

Choisissez un langage ésotérique existant, documenté sur esolangs.org. Écrivez-y **vous-même** un programme p original, qui calcule quelque chose de non trivial, et décrivez en langue naturelle ce qu'il calcule : c'est la spécification de p, que nous appelons spec ; elle peut aussi inclure des cas de test.

Le défi : faire en sorte que p ne soit pas générable par un LLM à partir de spec. Un LLM qui reçoit spec peut très bien produire un programme p' **sémantiquement** équivalent à p, qui passe vos tests ; ce qui compte, c'est qu'il ne produise pas p : la différence attendue est **syntaxique**. Pas de spec vague pour autant : spec reste assez précise pour qu'un autre étudiant, ou un LLM, puisse écrire un programme qui calcule la même chose que p.

Vous confronterez ensuite p à ce que le LLM produit, puis vous analyserez le résultat.

Contraintes

- **Langage existant.** Le langage doit avoir une page sur esolangs.org. Vous ne l'inventez pas.
- **Écrit par vous, sans LLM.** p ne doit pas avoir été généré, même en partie, par un LLM ou un agent de code. **Nous le vérifierons**, notamment en soumettant spec à des LLM. Si, par accident, un LLM produit un programme identique ou très proche de p, changez de couple (p, spec) : un autre calcul, ou un autre langage. Tout autre usage d'un outil d'IA (en dehors de la confrontation ci-dessous) est déclaré dans le `README.md`.
- **Programme original.** p ne doit pas se trouver déjà en ligne. Si vous vous inspirez d'un programme existant, citez-le.
- **Calcul non trivial.** Le résultat est calculé par p, pas écrit en dur dans le programme : afficher un texte fixe ne suffit pas. p peut lire une entrée, ou non. Exemples possibles, dont la difficulté dépend beaucoup du langage : PGCD de deux nombres, test de palindrome, conversion de base, nombres premiers jusqu'à n.
- **Exécutable.** p doit tourner sur un interpréteur ou un compilateur identifié, que nous pourrons relancer.
- **Expliqué.** p est commenté, et son fonctionnement est expliqué par écrit : vous devez pouvoir justifier chaque instruction.

Livrables

Un dépôt Git contenant les fichiers suivants (remplacez `ext` par l'extension usuelle du langage, par exemple `bf` pour Brainfuck) :

- `README.md` — le langage choisi (lien vers sa page esolangs.org), l'interpréteur ou le compilateur utilisé (lien, version), la commande qui lance p, et les points que la spécification du langage laisse ouverts : taille des cellules, débordement, comportement en fin d'entrée…
- `p.ext` — le programme p, commenté
- `EXPLICATION.md` — le fonctionnement de p : organisation de la mémoire, boucles, astuces
- `SPEC.md` — spec : la description en langue naturelle de ce que calcule p (format de l'entrée s'il y en a une, format de la sortie, cas particuliers), avec éventuellement des cas de test. Elle doit être assez précise pour qu'un autre étudiant, ou un LLM, puisse écrire sans vous poser de question un programme qui calcule la même chose que p. C'est ce texte que reçoit le LLM.
- `tests/` — les cas de test : `tests/01.in` (entrée) et `tests/01.out` (sortie attendue), puis `02`, `03`…, dont des cas limites, et d'autres que ceux de spec, que le LLM ne voit jamais. Si p ne lit pas d'entrée, `tests/01.out` seul.
- `run.sh` — exécute un programme (p ou une proposition p') sur tous les cas de test et affiche les résultats : `./run.sh p.ext`
- `llm/JOURNAL.md` — le journal de la confrontation au LLM, selon les règles ci-dessous
- `llm/p1.ext`, `llm/p2.ext`, … — les propositions p' du LLM, telles quelles
- `ANALYSE.md` — votre analyse du résultat
- `interpreteur/` — bonus : votre propre interpréteur

Confrontation au LLM

Le choix de l'outil est libre : LLM conversationnel ou agent de code, un ou plusieurs. Quatre règles s'appliquent.

1. **Même point de départ que vous.** Le LLM reçoit le nom du langage, sa page de spécification sur esolangs.org et spec (`SPEC.md`). Il ne voit jamais p.
2. **Conditions déclarées.** Indiquez tout ce que vous lui avez fourni en plus de spec : cas de test, interpréteur, possibilité d'exécuter du code, relances ou corrections de votre part.
3. **Mêmes tests.** Chaque proposition p' est exécutée sur vos cas de test, avec le même interpréteur que p (`run.sh`). Si vous avez donné des cas de test au LLM, gardez-en d'autres qu'il n'a jamais vus.
4. **Tout est tracé.** Le journal contient l'outil et le modèle (avec sa version), la date, les prompts exacts, les réponses ou la trace de session de l'agent, le nombre d'essais et les résultats des tests.

Analyse attendue

Le cœur de l'exercice est la distance syntaxique entre p et ce que produit le LLM : le but est un couple (p, spec), précis et non trivial, tel que le LLM, à partir de spec, ne génère pas p, même s'il en retrouve le calcul. Pour chaque proposition p', distinguez deux questions.

Sémantique : p' calcule-t-il la même chose que p ? Sur vos tests (`run.sh`), et au-delà : comment pourriez-vous le savoir ? Si aucune proposition ne passe vos tests, expliquez pourquoi selon vous : où le LLM se trompe-t-il ? Qu'est-ce qui, dans le langage ou dans le calcul, rend la tâche difficile ?

Syntaxe : p′ ressemble-t-il à p ? Comparez-les : structure, taille, organisation de la mémoire, algorithme, style. Mesurez la distance entre les deux textes, par exemple avec un `diff` ou une distance d'édition sur les instructions (commentaires et espaces ignorés), et justifiez la mesure choisie. Qu'est-ce qui, dans p, l'éloigne de ce qu'un LLM écrit spontanément ? Lequel des deux préférez-vous, et pourquoi ? Si une proposition est identique ou très proche de p, le défi n'est pas relevé : changez de couple (p, spec), comme indiqué dans les contraintes.

Concluez en quelques lignes : qu'est-ce que cette expérience vous apprend sur la distance entre ce qu'on veut calculer et le langage dans lequel on l'exprime ?

Modalités

- Travail : individuel.
- Date de rendu : jeudi 15 octobre 2026 à minuit (fin de journée).
- Format du rendu : un dépôt Git, organisé comme indiqué dans Livrables, accessible aux enseignants.
- Envoi : l'adresse du dépôt, par email à mathieu.acher@irisa.fr, avant la date de rendu.